

Parallel Text Search

Zwick, Patrick

patrick.zwick@htwg-konstanz.de

Döbele, Max

max.doebele@htwg-konstanz.de

Friesen, Marcel

marcel.friesen@htwg-konstanz.de

06 February 2026

Abstract

1 Introduction

2 Testing Data

For testing, we use a collection of up to 98 books from Project Gutenberg, which we concatenate into a single text containing over 82 million characters. We perform queries using, at most, the 100 most common English words. Both the number of books and the number of queries can be varied to evaluate performance under different conditions.

3 Sequential Implementations

3.1 Reference Implementation

To verify correctness, we use a simple baseline implementation, denoted `std`, which relies on `std::string::find()` for substring matching (see Listing 1).

```
std::vector<std::vector<size_t>>
find_std(const std::string &text, const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    for (size_t i = 0; i < queries.size(); ++i) {
        const std::string &query = queries[i];
        auto pos = text.find(query);
        while (pos != std::string::npos) {
            indices[i].push_back(pos);
            pos = text.find(query, pos + 1);
        }
    }
    return indices;
}
```

Listing 1: `find_std()` in `std_text_search.cpp`

3.2 Candidate Selection and Validation Approach

The first implementation is divided into two distinct phases:

1. Identification of candidate positions for potential occurrences based on a simple filtering criterion. A position is considered a candidate if three characters match: the first character of the query, the middle character, and the last character.
2. Verification of each candidate position to determine whether it constitutes a valid occurrence.

The initial approach, denoted `candidate_v1`, performs candidate identification followed by validation separately for each query (see Listing 2). The vector is cleared after each query and reused for the next, avoiding unnecessary allocations.

```

void find_candidates(const size_t text_length, std::vector<size_t> &results,
                    const std::string &text, const std::string &query) {
    const auto query_length = query.length();
    const auto mid = query_length >> 1;
    const auto end = query_length - 1;
    for (size_t i = 0; i < text_length - query_length; ++i) {
        if (text[i] == query[0] && text[i + mid] == query[mid] &&
            text[i + end] == query[end]) {
            results.push_back(i);
        }
    }
}

bool test_candidate(const size_t index, const std::string &text,
                   const std::string &query) {
    return std::memcmp(text.data() + index, query.data(), query.size()) == 0;
}

std::vector<std::vector<size_t>>
find_candidate_v1(const std::string &text,
                 const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    const auto text_length = text.length();
    std::vector<size_t> results;
    for (size_t i = 0; i < queries.size(); ++i) {
        const std::string &query = queries[i];
        find_candidates(text_length, results, text, query);
        for (const auto &result : results) {
            if (test_candidate(result, text, query)) {
                indices[i].push_back(result);
            }
        }
        std::memset(results.data(), 0, results.size() * sizeof(size_t));
    }

    return indices;
}

```

Listing 2: `find_candidates()`, `test_candidate()` and `find_candidate_v1()` in `candidate_v1_text_search.cpp`

3.2.1 Using an Array Instead of `std::vector`

Since a `std::vector` may need to repeatedly grow its underlying storage and copy existing elements when its capacity is exceeded, this can introduce a significant performance bottleneck, at least for the first query. To avoid this overhead, the `candidate_v2` implementation instead pre-allocates an array with a size equal to the length of the text. In the worst case, every position in the text could be a valid candidate, so this allocation is sufficient to store all candidates.

3.2.2 Using a Bitmask

To improve CPU cache utilization and reduce the memory footprint, we represent the candidate buffer as a *bitmask*, where each bit set to 1 denotes a valid candidate position. The bitmask is stored as an array of unsigned 64-bit integers, allowing 64 candidates to be packed into a single word.

Let m denote the length of the text. To represent candidate positions for a query of any length, we require

$$\left\lceil \frac{m + 63}{64} \right\rceil$$

64-bit words in the bitmask.

For a candidate at position k , the word index w of the corresponding 64-bit word B_w in the bitmask is obtained by dividing k by 64 which can be efficiently computed by a right shift of 6 bits:

$$w = \frac{k}{64} = k \gg 6.$$

The bit position b within the word B_w is given by taking k modulo 64, which can be efficiently computed by a bitwise AND with 63:

$$b = k \bmod 64 = k \& 63.$$

To set this bit, we construct a temporary value with only the target bit active by shifting the integer constant 1 to the left by the bit position b , and then update the mask word using the bitwise OR:

$$B_w \leftarrow B_w \vee (1 \ll b).$$

To reconstruct candidate positions from the bitmask, we iterate over all 64-bit words and extract the indices of the set bits.

For a given word index w , let B_w again denote the corresponding 64-bit word. If $B_w = 0$, the word contains no valid candidates. Otherwise, we repeatedly extract the least significant set bit until B_w becomes zero.

The position of this bit within the word is obtained using the number of trailing zeros:

$$b = \text{ctz}(B_w),$$

where ctz denotes the count of trailing zero bits.

The corresponding global candidate index is then

$$k = 64w + b.$$

After processing a candidate, the lowest set bit is cleared using the operation

$$B_w \leftarrow B_w \& (B_w - 1),$$

which removes the extracted bit and guarantees termination after all set bits have been processed.

This approach ensures that candidate extraction runs in time proportional to the number of valid candidates rather than the size of the bitmask.

3.2.3 Using a Bitmask for All Queries Together

3.3 Rolling-Hash Approach

3.4 Evaluation

4 Summary